《需求分析与系统设计》
Requirements Analysis and System Design

任课教师：范国祥

电　　话：0451-86418876-811(O)

13199561265(微信同号)

邮　　箱：fgx@hit.edu.cn

哈工大计算学部/

国家示范性软件学院

软件工程教研室

2023.09

UML图

主要内容

1. 用例图
2. 活动图
3. 类图/对象图
4. 时序图/协作图
5. 状态图
6. 组件图
7. 部署图

UML图：类图/对象图

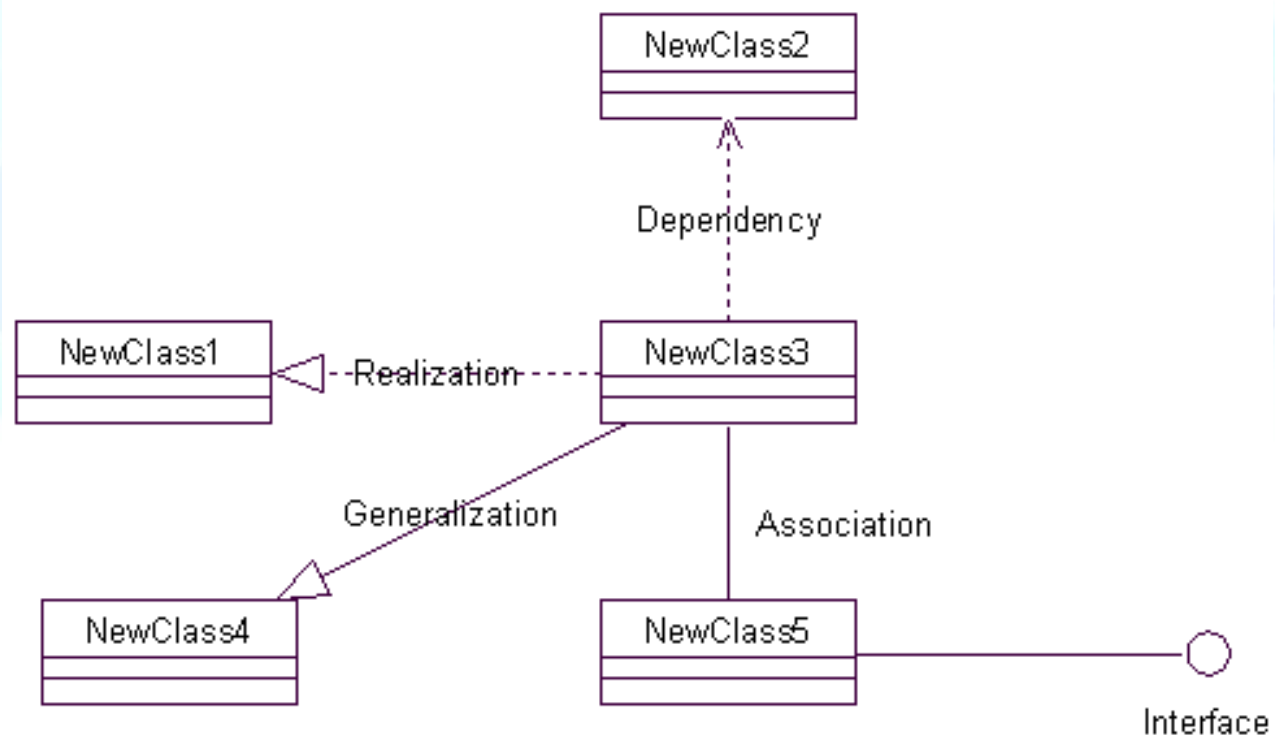
类图

- 类图的概念
- 类图的元素
- 类
- 接口
- 类之间的关系
- 分析阶段的类图
- 设计阶段的类图
- 实例：图书馆管理系统的类图

UML图：类图/对象图

类图的概念

- 描述类、接口及它们之间关系的图
- 显示系统中各个类的静态结构



UML图：类图/对象图

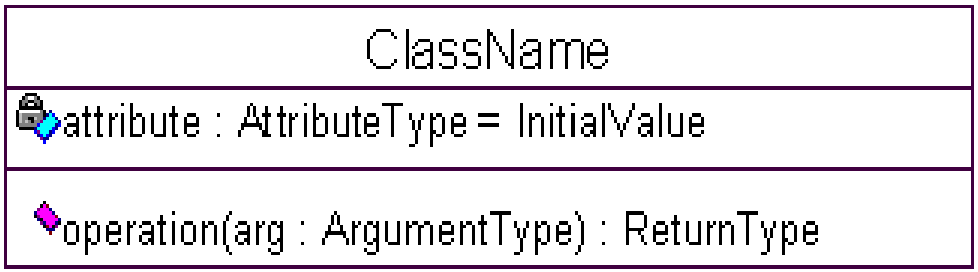
类图的元素

- 类（Class）
- 接口（Interface）
- 依赖关系（Dependency）
- 泛化关系（Generalization）/继承关系（Inheritance）
- 关联关系（Association）
- 实现关系（Realization）

UML图：类图/对象图

类

- 面向对象系统组织结构的核心
- 对一组具有相同属性、操作、关系和语义的对象的抽象
- 包括名称部分（Name）、属性部分（Attribute）和操作部分（Operation）



UML图：类图/对象图

类

- 类的组成
  - 名称
  - 属性
  - 操作

UML图：类图/对象图

类

- 名称
  - 应该是一个名词
  - 分为简单名称和路径名称
  - 每个单词首字母大写



简单名



路径名

UML图：类图/对象图

类

- 属性
  - 描述了类在软件系统中代表的事物（即对象）所具备的特性
  - 类可以有任意数目的属性，也可以没有属性
  - 在 UML 中，类属性的语法为

[可见性] 属性名 [类型] [=初始值]

UML图：类图/对象图

类

- 属性的组成
  - 可见性
  - 属性名
  - 类型
  - 初始值

UML图：类图/对象图

类

- 属性的可见性
  - 公有（Public）“＋”　　所有类可见
  - 私有（Private）“－”　　只对该类本身可见
  - 受保护（Protected）“＃”　该类及子孙可见
  - 包（Package）“～”　　只对同一包声明的类可见
- 属性名
  - 每个属性必须有一个名字以区别于类中的其他属性
  - 属性名由描述所属类的特性的名词或名词短语组成
  - 除了第一个单词外其余单词的首字母要大写

UML图：类图/对象图

类

- 属性的类型
  - 简单类型
    - 整型
    - 浮点型
    - 布尔型
    - 枚举类型
    - ……
  - SDK中的类、系统中的其他自定义类
- 属性的初始值
  - 保护系统的完整性，防止漏掉取值或被非法的值破坏系统的完整性
  - 为用户提供易用性

UML图：类图/对象图

类

- 操作
  - 对类的对象所能做的事务的抽象
  - 一个类可以有任意数量的操作或者根本没有操作
  - 返回类型、名称和参数一起被称为操作签名
  - 在 UML 中，类操作的语法为

[可见性] 操作名 [(参数列表)] [:返回类型]

UML图：类图/对象图

类

- 操作的组成
  - 可见性
  - 操作名
  - 参数表
  - 返回类型

UML图：类图/对象图

类

- 操作的可见性
  - 公有（Public） “＋”
  - 私有（Private） “－”
  - 受保护（Protected） “＃”
  - 包（Package） “～”
- 操作名
  - 用来描述所属类的行为的动词或动词短语
  - 命名规则与属性相同

UML图：类图/对象图

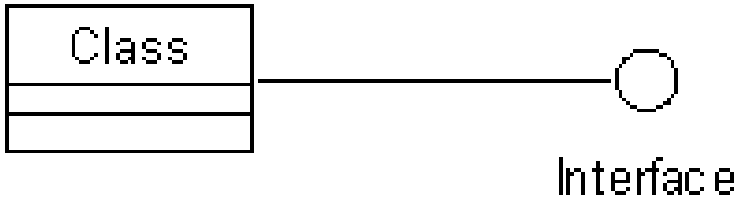
类

- 操作的参数表
  - 一些按顺序排列的属性定义了操作的输入
  - 是可选的，即操作不一定必须有参数才行
  - 定义方式：“名称：类型”
  - 若存在多个参数，将各个参数用逗号隔开
  - 参数可以具有默认值
- 操作的返回类型
  - 是可选的，即操作不一定必须有返回类型
  - 绝大部分编程语言只支持一个返回值
  - 具体的编程语言一般要加一个关键字 void 来表示无返回值

UML图：类图/对象图

接口

- 在没有给出对象的实现和状态的情况下对对象行为的描述
- 包含操作但不包含属性
- 没有对外界可见的关联
- 一个类可以实现一个或多个接口



UML图：类图/对象图

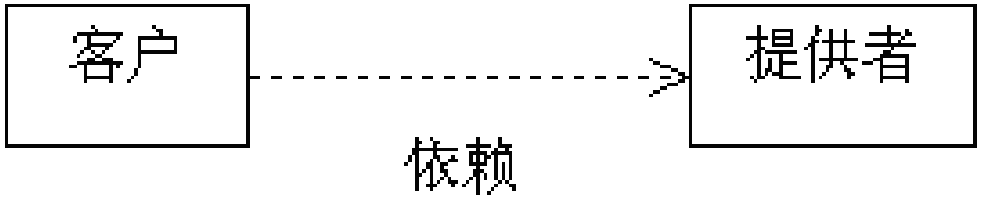
类之间的关系

- 依赖关系
- 泛化/继承关系
- 关联关系
- 实现关系

UML图：类图/对象图

类之间的关系

- 依赖关系
  - 表示两个或多个模型元素之间语义上的关系
  - 客户以某种形式依赖于提供者



UML图：类图/对象图

类之间的关系

- 依赖关系的分类
  - 使用依赖（Usage）
  - 抽象依赖（Abstraction）
  - 授权依赖（Permission）
  - 绑定依赖（Binding）

UML图：类图/对象图

类之间的关系

- 使用依赖
  - 表示客户使用提供者提供的服务以实现它的行为
  - 使用（《use》）
  - 使用依赖具体包括
    - 调用（《call》）：声明一个类调用其他类的操作的方法
    - 参数（《parameter》）：声明一个操作和它的参数之间的关系
    - 发送（《send》）：声明信号发送者和信号接收者之间的关系
    - 实例化（《instantiate》）：声明用一个类的方法创建了另一个类的实例

UML图：类图/对象图

类之间的关系

- *抽象依赖
  - 表示客户与提供者之间的关系，依赖于在不同抽象层次上的事物，包括
    - 跟踪（《trace》）
    - 精化（《refine》）
    - 派生（《derive》）

UML图：类图/对象图

类之间的关系

- *授权依赖
  - 表达一个事物访问另一个事物的能力，包括
    - 访问（《access》）：允许一个包访问另一个包的内容
    - 导入（《import》）：允许一个包访问另一个包的内容并为被访问包的组成部分增加别名
    - 友元（《friend》）：允许一个元素访问另一个元素，不管被访问的元素是否具有可见性

UML图：类图/对象图

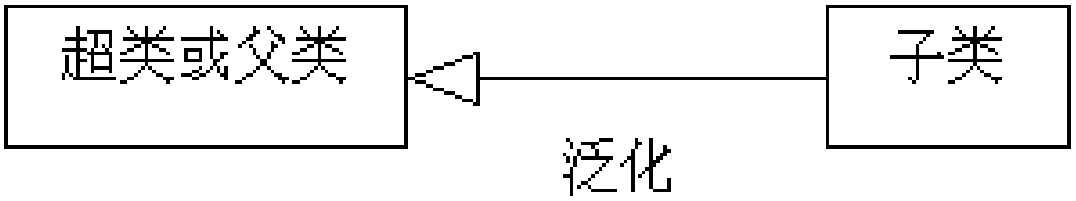
类之间的关系

- *绑定依赖
  - 较高级的依赖类型，用于绑定模板以创建新的模型元素，包括
    - 绑定（《bind》）

UML图：类图/对象图

类之间的关系

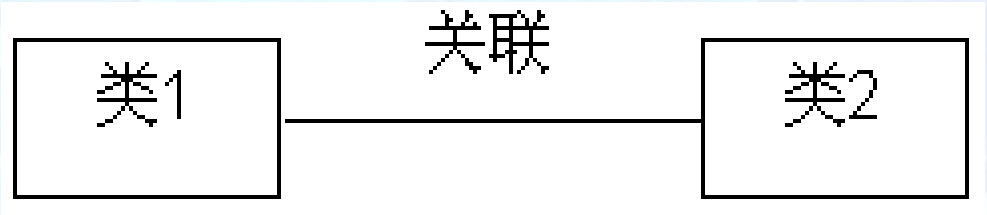
- 泛化/继承关系
  - 存在于一般元素和特殊元素间的分类关系
  - 可以用于类、用例以及其他模型元素
  - 描述了一种“is a kind of”的关系



UML图：类图/对象图

类之间的关系

- 关联关系
  - 一种结构关系
  - 指明事物的对象之间的联系



UML图：类图/对象图

类之间的关系

- 关联关系
  - 名称（Name）
  - 角色（Role）
  - 多重性（Multiplicity）
  - 聚合关系（Aggregation）
  - 组合关系（Composition）
  - 导航性（Navigation）

UML图：类图/对象图

类之间的关系

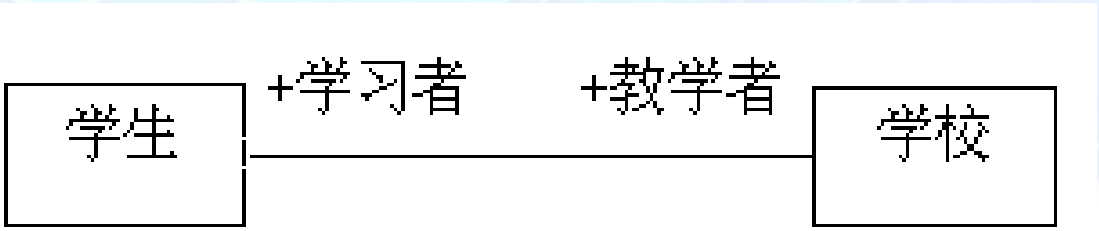
- 关联的名称
  - 使用一个动词或动词短语来命名关联
  - 清晰而简洁地说明对象间关系
  - 关联的名称并不是必需的
  - 可以前缀或后缀一个指引阅读方向的方向指示符，以消除歧义



UML图：类图/对象图

类之间的关系

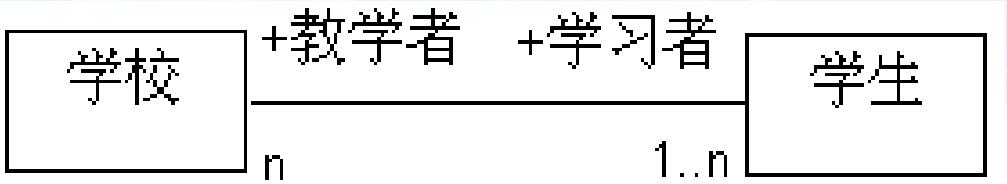
- 关联的角色
  - 关联关系中一个类对另一个类所表现出来的职责
  - 角色的名称应该是名词或名词短语，以解释对象是如何参与关系的



UML图：类图/对象图

类之间的关系

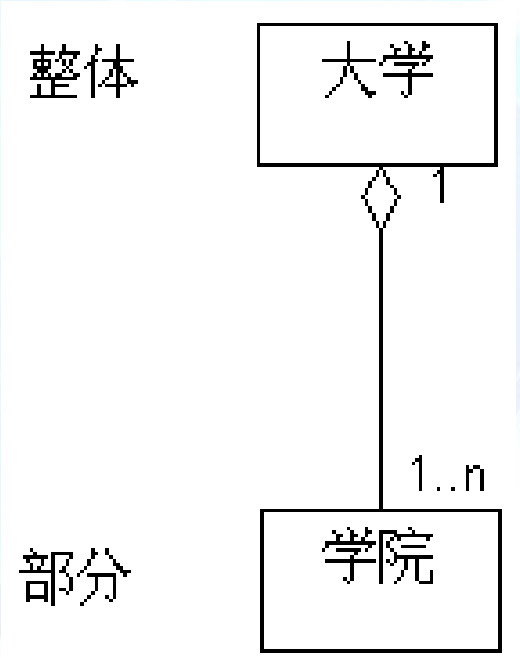
- 关联的多重性
  - 指有多少对象可以参与该关联
  - 可以表达一个取值范围、特定值、无限定的范围或一组离散值
  - 格式：“min .. max”（均为int型）
  - 赋给一个端点的多重性表示该端点可以有多少个对象与另一个端点的一个对象关联



UML图：类图/对象图

类之间的关系

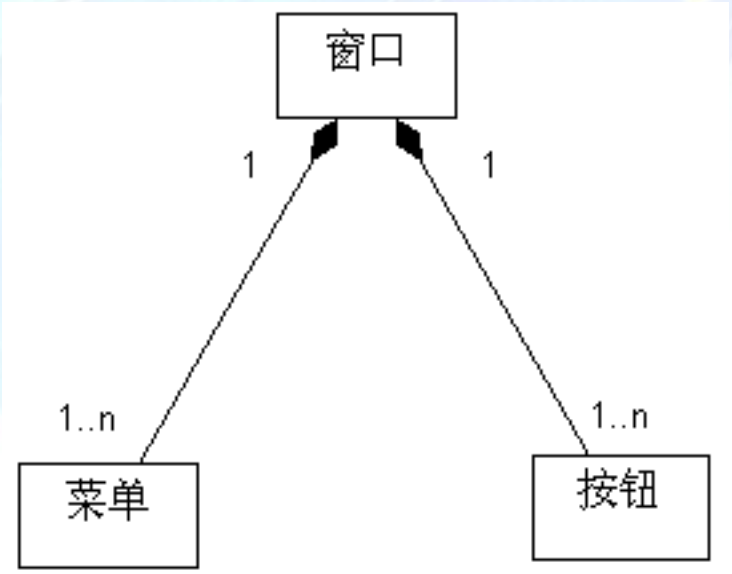
- 聚合关系
  - 一种特殊类型的关联
  - 表示整体与部分关系的关联
  - 描述了“has a”的关系



UML图：类图/对象图

类之间的关系

- 组合关系
  - 组合是聚合关系中的一种特殊情况，是更强形式的聚合，又称强聚合
  - 成员对象的生命周期取决于聚合的生命周期
  - 聚合不仅控制着成员对象的行为，而且控制着成员对象的创建和析构



UML图：类图/对象图

类之间的关系

- 实现关系
  - 规格说明（接口）和其实现之间的关系
  - 客户必须至少支持提供者的所有操作
  - 泛化和实现都可以将一般描述与具体描述联系起来
  - 泛化将同一语义层上的元素连接起来，并且通常在同一模型内
  - 实现将不同语义层内的元素连接起来，并且通常建立在不同的模型内



UML图：类图/对象图

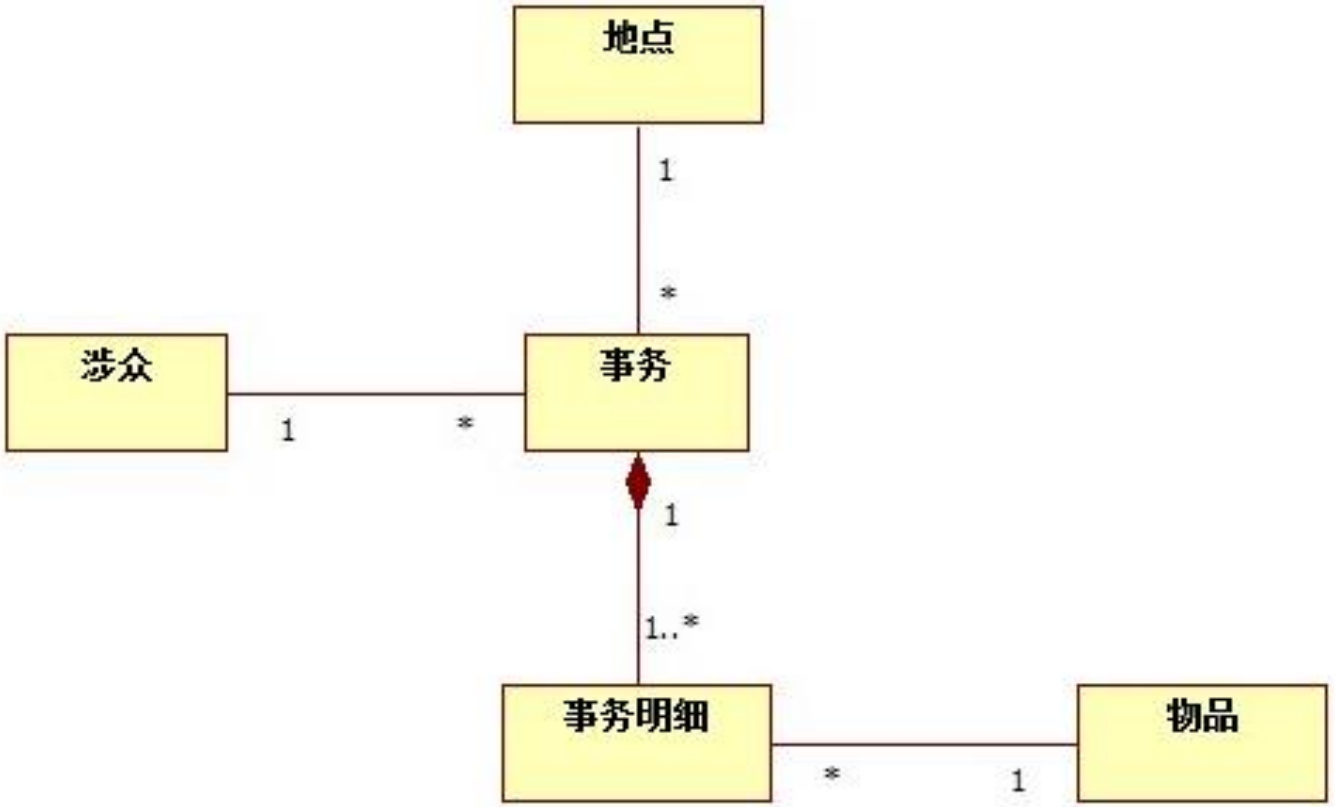
分析阶段的类图

- 分析阶段主要对领域建模
- 分析阶段的类图与实现技术和平台无关，分析师掌握如下概念即可
  - 类
    - 属性
    - 操作
    - 可见性
      - 私有
      - 公有
  - 关联关系

UML图：类图/对象图

分析阶段的类图

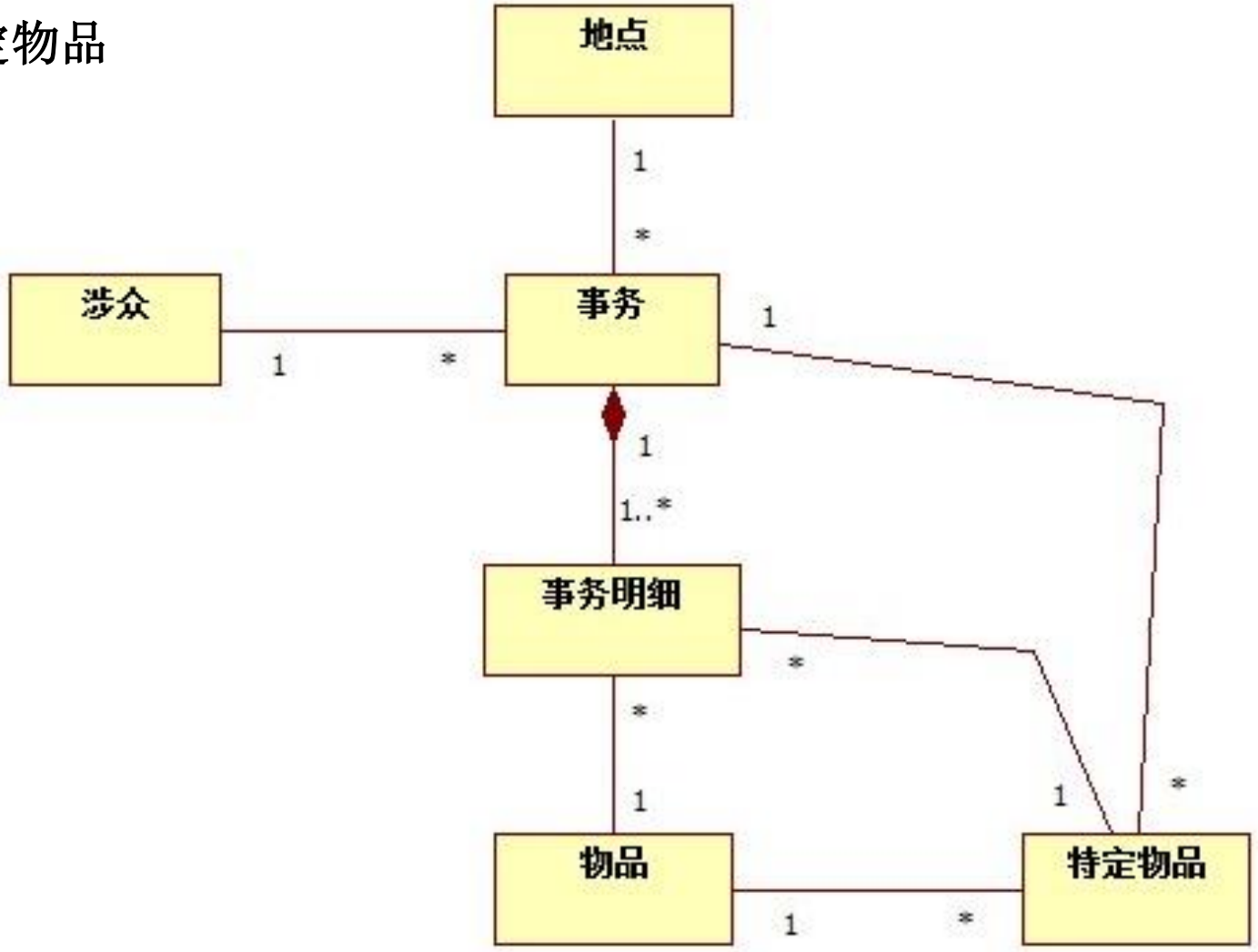
- 使用“事务模式（Transaction Pattern）”对领域建模
  - 事务模式由OOAD大师Peter Coad提出



UML图：类图/对象图

分析阶段的类图

- 物品与特定物品



UML图：类图/对象图

分析阶段的类图

- 后续事务

